Trabalho Prático III – Introdução a Bancos de Dados

Armazenamento, índices, otimizador de consultas e transações no PostgreSQL

Ruthelene Rodrigues Farias
Matrícula 22552011
UFAM / IComp – 2026/02

Novembro de 2026

Visão geral

Todos os resultados deste relatório foram produzidos pelos programas do repositório (`src/t01_sistema.py` a `src/t17_reservas.py`), executados por `make parteI`, `parteII`, `parteIII` e `parteIV` sobre PostgreSQL 16 em contêiner Docker, no GitHub Codespaces. Cada programa grava um arquivo JSON em `resultados/`, e as tabelas da Seção 9.3 foram extraídas desses arquivos. Os parâmetros individuais, calculados a partir da matrícula, estão na Tabela 1.

Tabela 1: Parâmetros individuais (`params.json`).

Parâmetro	Valor
Matrícula	22552011
Fator de escala TPC-H (`sf`)	0,1
Consultas TPC-H	1, 3, 5, 11, 14, 19, 20, 21
`qgen_seed`	567059
`t_seed`	-0,980958
Valores distintos de `v`	100
`fillfactor`	70, 85, 95, 100
Filme-alvo (T14)	American Beauty
Votos baixo / alto (T12, T16)	1.000 / 35.000
Letras de prefixo / sufixo (T15)	C / I
Assentos / espera de escolha (T17)	120 / 0,1 s
Agentes `k`	1, 2, 4, 6, 8, 10

9.1 Decisões de projeto

Organização. Cada tarefa é um programa independente que lê `params.json`, executa o experimento e grava seu JSON por uma função comum. As partes são executadas em ordem pelo `Makefile`, e o verificador (`make test`) refaz medições de forma independente e compara com o que foi registrado. Na última execução do verificador, as 157 verificações passaram.

Parte I. Na T2 foi criado um arquivo temporário de 256 MiB, lido em blocos de 8 KiB com `O_DIRECT` (contorna o cache de páginas do sistema operacional): uma passada sequencial sobre os 32.768 blocos e 20.000 leituras em posições sorteadas (semente fixa). A razão é o tempo por bloco aleatório dividido pelo tempo por bloco sequencial. Na T4 cada consulta foi executada uma única vez com `EXPLAIN (ANALYZE, BUFFERS)`; portanto, os blocos "read" refletem execução com cache

parcialmente frio. O "acesso dominante" é uma heurística: a varredura que processa mais linhas no plano; o plano completo está em `resultados/t04.json`.

Parte II. A tabela `t` é criada com a função do template (semente individual). Os índices foram criados com `max_parallel_maintenance_workers = 0`. A consulta medida é a igualdade `v = 50` (metade de $C = 100$). Na T7 o paralelismo de consulta foi desligado, os contadores de `t` foram zerados com `pg_stat_reset_single_table_counters` e o *flush* das estatísticas foi forçado antes de ler `pg_statio_user_tables`; a mesma execução fornece os blocos de `EXPLAIN (BUFFERS)`. O mesmo código serve a T9 e a T10, que só diferem pela ordem do índice (`v DESC NULLS FIRST`) e pela lista de *fillfactors*.

Parte III. Consultas de T12: seletiva `votes > 35000` e não seletiva `votes > 1000`. O limiar foi procurado com `EXPLAIN` (sem executar) para `votes < X`, com X crescendo de 50 em 50, até o primeiro plano `Seq Scan`. Em T11, o número de chaves por folha vem de `bt_page_stats`; os blocos de dados por chave são medidos pelo `ctid`; já os blocos-folha por chave são uma *estimativa* (max(1, folhas/chaves distintas)). Na T15 os índices criados são `title text_pattern_ops`, `substr(title,1,1)` e `reverse(title) text_pattern_ops`, seguidos de `ANALYZE` (necessário para haver estatísticas das expressões); o programa também registra o plano com `enable_seqscan = off` para mostrar se o índice seria possível. Na T16 o `STATISTICS` de `votes` é restaurado ao padrão no final.

Parte IV: reservas. A tabela é `assentos(num_voo, disp)`, em que `num_voo` identifica cada um dos 120 assentos (o enunciado fixa duas colunas, então a coluna `num_voo` faz o papel de identificador do assento). O passo 3 é `UPDATE assentos SET disp = false WHERE num_voo = X AND disp = true`. *A garantia de que nenhum assento é reservado duas vezes vem dessa condição, e não do nível de isolamento.* O conflito é detectado de três formas: (i) o `UPDATE` não altera linha (o assento já foi tomado); (ii) falha de serialização (SQLSTATE 40001); (iii) *deadlock* (40P01). Nos três casos faz-se `ROLLBACK` e uma nova tentativa completa (novo passo 1, nova escolha aleatória), e o número de tentativas de cada cliente é registrado. Cada agente é uma *thread* com sua própria conexão; os 120 clientes ficam numa fila compartilhada; o relógio só inicia quando todos os agentes estão conectados (barreira) e termina quando o último cliente é atendido. As reservas são gravadas em `reservas_log` depois da medição, e o programa aborta se houver assento repetido, cliente sem reserva ou número de assentos ocupados diferente de 120. Na versão *a* os três passos formam uma só transação; na versão *b*, o passo 1 é uma transação, a espera de 0,1 s ocorre fora de transação, e o passo 3 é outra transação.

9.2 Previsão antes da medição

As previsões foram escritas em `docs/previsao.md` e commitadas antes de qualquer arquivo de `resultados/` (commit `0cde786`; o commit `4f1310a` apenas removeu uma linha residual de formatação, sem mudar as previsões). O verificador confirma a ordem ("previsao.md versionado antes de resultados/"). Resumo do que foi previsto:

Tabela 2: Resumo das previsões registradas.

Item	Previsão
Páginas de `t`	443 páginas, 226 tuplas por página (36 B por tupla com ponteiro; 8168 B úteis por página)
Altura do índice	1 para 100.000 tuplas (cerca de 90 páginas); 2 para 1.000.000 (850–900 folhas), graças à deduplicação ($C = 100$)
Fillfactor	tamanho $70 > 85 > 95 > 100$; altura igual; consulta praticamente igual; criação um pouco mais lenta com *fillfactor* baixo; índice descendente igual ao crescente
T12	seletiva: Index/Bitmap Scan; não seletiva: Seq Scan; limiar entre 5% e 20% das linhas
T15	`LIKE 'C%'`, `substr` e `reverse` passam a usar índice; `LIKE '%I'` continua em Seq Scan
T16	erro pequeno em `votes < 1000`; erro relativo maior em `votes > 35000`; `STATISTICS 1000` aproxima da realidade
T17	a/RC: cai com k, menos que $1/k$; a/SERIALIZABLE: pouca melhora ou piora; b/RC: quase $1/k$; b/SERIALIZABLE: perto de b/RC, um pouco acima; b mais rápida que a

9.3 Medição

Parte I

Tabela 3: T1 – identificação do sistema (ambiente do GitHub Codespaces).

Item	Valor
CPU	AMD EPYC 7763 64-Core Processor (2 núcleos visíveis)
Memória RAM	7.942 MB
Disco (capacidade do sistema de arquivos)	33,6 GB
Caches	L1d 32 KiB; L1i 32 KiB; L2 512 KiB; L3 32 MiB
Sistema operacional / kernel	Debian GNU/Linux 13 (trixie) / 6.8.0-1064-azure
PostgreSQL	16.15
`block_size`	8192 B
`shared_buffers` / `effective_cache_size`	128MB / 4GB
`seq_page_cost` / `random_page_cost`	1 / 4
`work_mem` / `default_statistics_target`	4MB / 100

Tabela 4: T2 – armazenamento.

Item	Valor
Bloco do sistema de arquivos (`stat -f` / `stat`)	4096 B / 4096 B
Bloco do PostgreSQL	8192 B
Modo de leitura / arquivo de teste	O_DIRECT / 256 MiB
Leitura sequencial	32.768 blocos de 8 KiB: 41,5 µs/bloco (188,4 MiB/s)
Leitura aleatória	20.000 blocos: 35,2 µs/bloco (28.422 IOPS)
Razão custo aleatório / sequencial	0,85

Tabela 5: T3 – TPC-H carregado com `sf` = 0,1.

Tabela	Linhas	Tabela (MiB)	Total com índices (MiB)
`region`	5	0,01	0,02
`nation`	25	0,01	0,04
`part`	20.000	3,25	3,73
`supplier`	1.000	0,34	0,43
`partsupp`	80.000	13,69	16,95
`customer`	15.000	2,82	3,31
`orders`	150.000	20,69	25,18
`lineitem`	600.572	88,19	114,41

Tabela 6: T4 – consultas TPC-H com `EXPLAIN (ANALYZE, BUFFERS)` (uma execução de cada).

Consulta	Acesso dominante	Junções	Custo est.	Linhas est./reais	Blocos hit/read	Tempo (ms)
q01	Seq Scan em lineitem	–	24.014,63	6 / 4	2.372 / 8.930	658,4
q03	Seq Scan em orders	Hash Join, Nested Loop	22.478,73	31.680 / 1.164	53.105 / 8.772	189,1
q05	Seq Scan em orders	Hash Join, Nested Loop	12.817,73	25 / 5	23.056 / 850	49,4
q11	Index Scan em partsupp	Nested Loop	1.045,77	1.067 / 18	8.698 / 40	15,3
q14	Seq Scan em part	Hash Join	16.979,21	1 / 1	10.596 / 2.062	95,9
q19	Seq Scan em part	Nested Loop	7.066,14	1 / 1	1.656 / 289	14,5
q20	Index Scan em lineitem	Hash Join, Nested Loop	8.853,21	11 / 20	9.226 / 1.537	26,8
q21	Seq Scan em lineitem	Hash Join, Nested Loop	18.189,44	1 / 34	67.308 / 3.315	183,5

Parte II

Tabela 7: T5, T6 e T7 – tabela `t` (100.000 tuplas).

Item	Valor
T5 – soma de `v` / tuplas	4.955.172 / 100.000
T6 – `relpages` / `reltuples`	443 / 100.000
T6 – bytes / `block_size` = páginas	3.629.056 / 8192 = 443
T6 – tuplas por página (média)	225,73
T7 – `pg_stats` de `k`	n_distinct = -1; correlação = 1,00; histograma com 100 faixas
T7 – `pg_stats` de `v`	n_distinct = 100; correlação = 0,006; 100 valores comuns (freq. entre 0,85% e 1,15%); sem histograma
T7 – consulta `v = 50` (sem índice)	Seq Scan; 443 blocos em memória e 0 lidos (pg_statio); EXPLAIN: 443 / 0

Tabela 8: T8, T9 e T10 – índice B-tree sobre `v` (consulta `v = 50`).

Tarefa	Configuração	Págs.	Alt.	Folhas	Dens. (%)	Criação (ms)	Consulta (ms)	Plano
T8 (cresc.)	100.000 tuplas, ff 90 (padrão)	90	1	88	86,29	66	0,41	Bitmap Heap Scan
T8 (cresc.)	1.000.000 tuplas, ff 90 (padrão)	854	2	847	89,50	999	4,55	Bitmap Heap Scan
T9 (cresc.)	1.000.000, ff 70	1084	2	1076	70,54	1.206	5,04	Bitmap Heap Scan
T9 (cresc.)	1.000.000, ff 85	854	2	847	89,50	1.064	4,49	Bitmap Heap Scan
T9 (cresc.)	1.000.000, ff 95	854	2	847	89,50	1.290	5,26	Bitmap Heap Scan
T9 (cresc.)	1.000.000, ff 100	854	2	847	89,50	892	4,35	Bitmap Heap Scan
T10 (desc.)	1.000.000, ff 100	854	2	847	89,50	943	4,82	Bitmap Heap Scan
T10 (desc.)	1.000.000, ff 70	1084	2	1076	70,54	1.211	7,13	Bitmap Heap Scan
T10 (desc.)	1.000.000, ff 85	854	2	847	89,50	883	6,41	Bitmap Heap Scan
T10 (desc.)	1.000.000, ff 95	854	2	847	89,50	1.012	6,14	Bitmap Heap Scan
T10 (desc.)	1.000.000, ff 100	854	2	847	89,50	1.261	6,20	Bitmap Heap Scan

Parte III

Tabela 9: T11 – índices de `movie` (1.844 linhas em 15 blocos).

Índice	Atributo	Alt.	Folhas	Chaves/folha (máx./média)	Distintas	Linhas/chave	Blocos de dados/chave
`movie_key`	`id`	1	5	380 / 369,6	1.844	1,000	1,000
`movie_title`	`title`	1	10	238 / 185,0	1.833	1,006	1,005
`movie_votes`	`votes`	1	9	214 / 169,2	1.481	1,245	1,003

Tabela 10: T12 – consultas por intervalo. Limiar: `votes < X` passa a usar Seq Scan em X = 1.550 (677 linhas, 36,7% da tabela).

Consulta	Linhas	Plano	Custo	Blocos (hit)	Tempo (ms)
Seletiva: `votes > 35000`	6	Index Scan	8,49	3	0,015
Não seletiva: `votes > 1000`	1.516 (82,2%)	Seq Scan	38,05	15	0,353

Tabela 11: T13 e T14 – consultas equivalentes (T13: resultado = título devolvido; T14: American Beauty).

Consulta	Linhas	Nós do plano	Custo	Tempo (ms)
T13 – `MAX`	Star Wars	Index Scan, Result, Limit, Index Only Scan	35,37	0,110
T13 – `>= ALL`	Star Wars	Seq Scan, Materialize, Seq Scan	43.620,99	1,239
T14 – subconsulta	9	Index Scan, Index Scan	43,34	0,096
T14 – auto-junção	9	Nested Loop, Index Scan, Index Scan	49,49	0,025

Tabela 12: T15 – planos antes e depois de criar os índices `text_pattern_ops`, sobre `substr` e sobre `reverse`.

Consulta	Linhas	Antes (custo)	Depois (custo)
`prefixo_like`	130	Seq Scan (38,05)	Bitmap Heap Scan, Bitmap Index Scan (22,19)
`prefixo_substr`	130	Seq Scan (42,66)	Bitmap Heap Scan, Bitmap Index Scan (22,23)
`sufixo_like`	26	Seq Scan (38,05)	Seq Scan (38,05)
`sufixo_reverse`	26	Seq Scan (42,66)	Bitmap Heap Scan, Bitmap Index Scan (20,53)

Tabela 13: T16 – estimativa de seletividade sobre `movie.votes`.

Consulta	Linhas reais	Estimadas (padrão, 100 faixas)	Estimadas (STATISTICS 1000)
`votes < 1000`	326	328	325
`votes > 35000`	6	12	5

Parte IV

Tabela 14: T17 – reservas (120 assentos, 120 clientes). Speedup = tempo com $k = 1$ dividido pelo tempo com k agentes.

Ver.	Isolamento	k	Tempo (s)	Speedup	Tent. média	Tent. máx.	Tent. total	Assento tomado	Falha 40001	Deadlock
a	read committed	1	12,28	1,00	1,00	1	120	0	0	0
a	read committed	2	6,36	1,93	1,02	2	123	3	0	0
a	read committed	4	3,50	3,51	1,10	3	132	12	0	0
a	read committed	6	2,47	4,97	1,11	4	133	13	0	0
a	read committed	8	2,05	5,99	1,21	5	145	25	0	0
a	read committed	10	1,65	7,44	1,22	3	146	26	0	0
a	serializable	1	12,26	1,00	1,00	1	120	0	0	0
a	serializable	2	12,28	1,00	2,00	120	240	0	120	0
a	serializable	4	12,42	0,99	3,98	118	478	0	358	0
a	serializable	6	12,46	0,98	5,93	119	712	0	592	0
a	serializable	8	12,51	0,98	7,87	121	944	0	824	0
a	serializable	10	12,62	0,97	9,82	124	1.178	0	1058	0
b	read committed	1	12,33	1,00	1,00	1	120	0	0	0
b	read committed	2	6,38	1,93	1,02	2	123	3	0	0
b	read committed	4	3,50	3,52	1,10	4	132	12	0	0
b	read committed	6	2,58	4,78	1,18	3	141	21	0	0
b	read committed	8	1,96	6,28	1,18	3	142	22	0	0
b	read committed	10	1,69	7,29	1,18	4	141	21	0	0
b	serializable	1	12,31	1,00	1,00	1	120	0	0	0
b	serializable	2	8,21	1,50	1,32	4	158	4	34	0
b	serializable	4	5,65	2,18	1,79	8	215	10	85	0
b	serializable	6	3,49	3,53	1,57	6	189	16	53	0
b	serializable	8	2,98	4,13	1,82	10	218	19	79	0
b	serializable	10	3,42	3,60	2,46	11	295	22	153	0

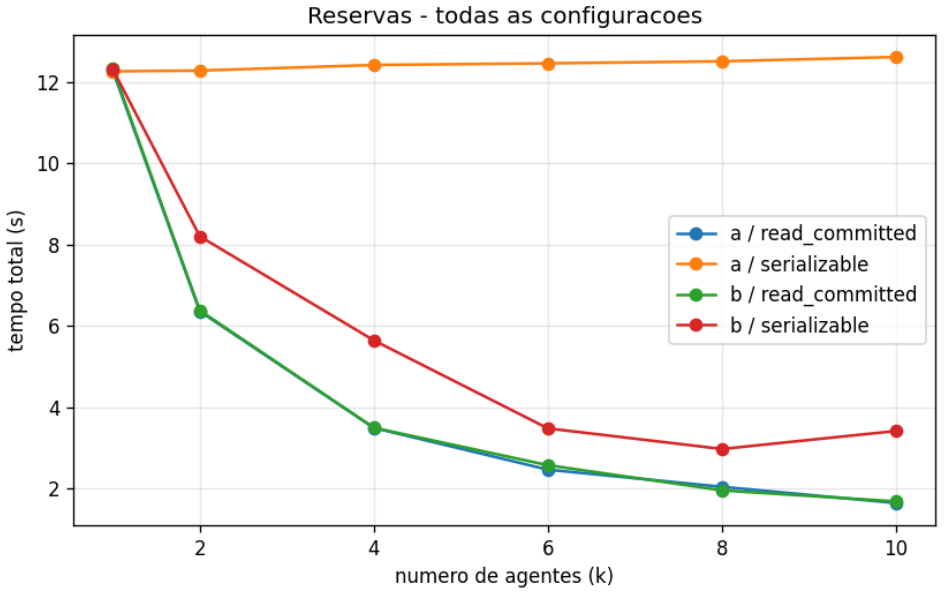


Figura 1: T17 – tempo total em função do número de agentes k, nas quatro configurações.

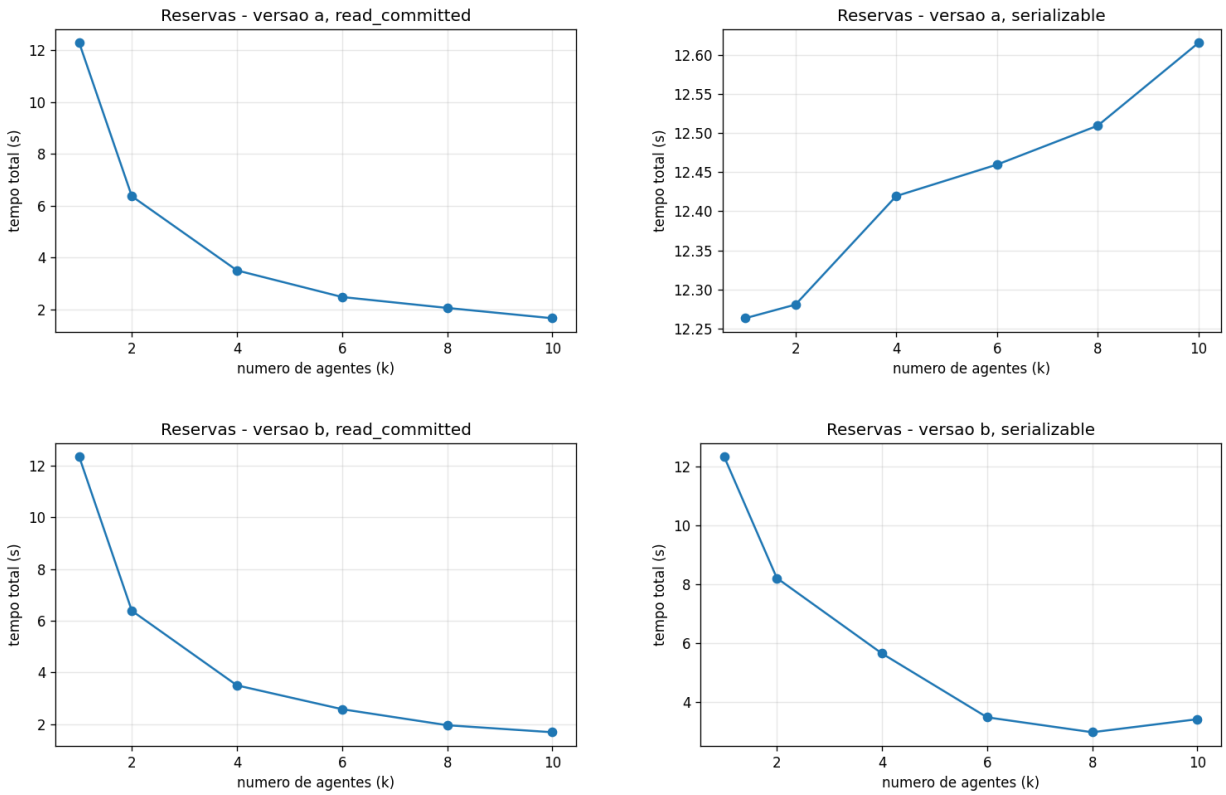


Figura 2: T17 – gráficos individuais (versão a em cima, versão b embaixo; READ COMMITTED à esquerda, SERIALIZABLE à direita).

9.4 Comparação e explicação das divergências

Tabela 15: Previsto × medido.

Item	Previsto	Medido	
Páginas de `t`	443; 226 tuplas/pág.	443; 225,73 (média)	✓
Altura, 100.000	1 (cerca de 90 págs.)	1 (90 págs., 88 folhas)	✓
Altura, 1.000.000	2 (850–900 folhas)	2 (847 folhas)	✓
Fillfactor: tamanho	$70 > 85 > 95 > 100$	só o 70 difere (1084 págs.); 85, 95 e 100 = 854	divergiu
Fillfactor: criação	ligeiramente maior com *ff* baixo	883–1290 ms, sem tendência	inconclusivo
Fillfactor: consulta	praticamente igual	4,35–5,26 ms	✓
Descendente	igual ao crescente	idêntico em páginas, folhas, altura	✓
T12 seletiva / não seletiva	Index Scan / Seq Scan	Index Scan (6 linhas) / Seq Scan (82,2%)	✓
T12 limiar	5% a 20%	36,7% ($X = 1550$)	divergiu
T15	3 passam a usar índice; sufixo não	exatamente isso (Bitmap Index Scan)	✓
T16	erro maior na cauda; mais estatística ajuda	328 vs 326; 12 vs 6; com 1000: 325 e 5	✓
T17 a/RC	cai com k, menos que $1/k$	12,28 s → 1,65 s (speedup 7,4)	✓
T17 a/SER	pouca melhora ou piora	constante, 12,3–12,6 s; tentativas $\approx k$	✓(mais extremo)
T17 b/RC	quase $1/k$	12,33 s → 1,69 s (speedup 7,3)	✓
T17 b/SER	perto de b/RC	3,42 s com $k = 10$, o dobro de b/RC	divergiu
T17 a × b em RC	b mais rápida que a	praticamente iguais	divergiu

D1. *Fillfactor*: só o valor 70 mudou o tamanho do índice

Como $C = 100$, cada valor de `v` aparece cerca de 10.000 vezes em 1.000.000 de tuplas, e o PostgreSQL 16 *deduplica* entradas repetidas: guarda uma única tupla de índice com uma lista de ponteiros (6 B por ponteiro). Na construção do índice o tamanho dessa tupla é limitado a cerca de 10% da página (algo como 800 B, aproximadamente 130 ponteiros). Assim, uma folha guarda poucas tuplas grandes, e o *fillfactor* só pode mudar o conteúdo da folha em passos de uma dessas tuplas. A densidade medida é compatível com isso: 89,5% corresponde a 9 tuplas por folha e 70,5% a 7 tuplas. Com 9 tuplas × cerca de 130 ponteiros ≈ 1.200 ponteiros por folha, 1.000.000 de linhas ocupam algo entre 830 e 850 folhas, compatível com as 847 medidas. Os valores 85, 90 (padrão), 95 e 100 caem no mesmo número de tuplas por folha; só o 70 desce para 7, e então o índice cresce para 1076 folhas (cerca de 27% a mais). Esta é uma *hipótese coerente com os números*, não verificada diretamente (por exemplo, com `pageinspect` ou criando o índice com `deduplicate_items = off`, caso em que se esperaria tamanho monotônico em relação ao *fillfactor*).

D2. Tempos de criação e de consulta

Cada configuração foi medida uma única vez. Os tempos de criação (883 a 1290 ms) variam sem relação monotônica com o *fillfactor* (o 95 foi mais lento que o 70), então a diferença prevista, que é pequena, não é distinguível do ruído de uma medição única em máquina virtual compartilhada. Os tempos de consulta de T9 (4,35 a 5,26 ms) e de T10 (4,82 a 7,13 ms) também ficam na mesma faixa, e o plano é o mesmo em todos os casos.

D3. Limiar de T12 acima do previsto (36,7% em vez de 5–20%)

A previsão supunha o comportamento de tabelas grandes, em que o acesso aleatório às páginas domina o custo do índice. A tabela `movie` tem só 15 blocos. O custo do `Seq Scan` é praticamente fixo: $15 \times 1 + 1.844 \times (0{,}01 + 0{,}0025) = 38{,}05$, exatamente o custo medido nas consultas T12 e T15. O plano por índice lê no máximo 15 blocos de dados (o número de blocos buscados satura), e seu custo cresce sobretudo com o número de linhas devolvidas (custos de CPU por tupla). Por isso o ponto de cruzamento só ocorre quando a consulta devolve uma fração grande da tabela. A previsão de 5–20% valeria para uma tabela com milhares de páginas.

D4. Versões a e b praticamente iguais em READ COMMITTED

Eu previ a versão b mais rápida. Em READ COMMITTED, uma transação só segura travas a partir do `UPDATE` (a leitura do passo 1 não bloqueia ninguém), e a espera de 0,1 s da versão a acontece *antes* do `UPDATE`. Ou seja, nenhuma das duas versões segura travas durante a espera, e ambas leem a lista de assentos antes de esperar. Os conflitos vêm só de agentes que sorteiam o mesmo assento, o que ocorre com a mesma frequência nas duas.

D5. b/SERIALIZABLE bem acima de b/READ COMMITTED

No nível SERIALIZABLE o PostgreSQL usa Serializable Snapshot Isolation (SSI), que aborta com 40001 quando detecta uma estrutura perigosa de dependências de leitura-escrita. Mesmo na versão b houve 34 a 153 falhas por configuração. Causas plausíveis (não separadas na medição): (i) quando dois agentes atualizam o mesmo assento, o SERIALIZABLE aborta o segundo com 40001, enquanto no READ COMMITTED o segundo reavalia a condição `disp = true` e simplesmente conta como "assento tomado"; (ii) as travas de predicado (SIREAD) da leitura de toda a tabela no passo 1 podem permanecer depois do *commit*, enquanto houver transações concorrentes, e participar de conflitos com escritas posteriores. O tempo piorou de $k = 8$ (2,98 s) para $k = 10$ (3,42 s), junto com mais falhas (79 para 153), mas cada ponto é uma única execução e há variação entre execuções.

9.5 Análises por parte

Parte I

Tipo de acesso e custo. As consultas q01, q03, q05, q14, q19 e q21 têm uma varredura sequencial como acesso dominante, em geral com *hash join*, e q11 e q20 têm acesso por índice com *nested loop*. A q01 lê praticamente toda a `lineitem` (11.302 blocos acessados, o número de páginas da tabela) e devolve só 4 grupos, daí o maior custo (24.014,63) e o maior tempo (658,4 ms). As consultas com filtros seletivos sobre dimensões pequenas (q11 e q20) usam índice e têm os custos mais baixos (1.045,8 e 8.853,2). A ordenação por custo e por tempo coincide em seis das oito posições (q01, q03, q21, q14, q05, q20); só q19 e q11, com tempos quase iguais (14,5 e 15,3 ms), trocam de lugar. O custo é uma unidade abstrata (1 = leitura sequencial de uma página), não milissegundos: o custo da q01 é cerca de 23 vezes o da q11, e o tempo, cerca de 43 vezes. As estimativas de linhas erram bastante: q03 estimou 31.680 linhas e devolveu 1.164; q11, 1.067 contra 18; q21, 1 contra 34. Como cada consulta foi executada uma vez, os blocos "read" mostram o efeito de cache frio (q01 leu 8.930 de 11.302 blocos fora do `shared_buffers`).

Tamanho de bloco. No PostgreSQL o bloco (8 KiB, medido em `block_size`) é definido na compilação (`-with-blocksize`, de 1 a 32 KiB, potência de 2) e fixado quando o *cluster* é criado com `initdb`; para mudá-lo é preciso recompilar, criar um novo *cluster* e recarregar os dados (`pg_dump`/restauração). No sistema de arquivos o bloco (4 KiB aqui, tanto em `stat -f` quanto

em `stat`) é definido ao formatar a partição (por exemplo, `mkfs`), e em Linux não pode exceder o tamanho da página de memória (4 KiB) para ser montado. Dentro de um contêiner o sistema de arquivos é o do hospedeiro, então o bloco não pode ser alterado de dentro do contêiner. Blocos maiores reduzem o número de operações em leituras sequenciais, mas desperdiçam leitura e escrita em acessos pontuais.

Custo aleatório/sequencial medido. Medimos razão 0,85 (35,2 µs por bloco aleatório contra 41,5 µs por bloco sequencial), enquanto o PostgreSQL assume `random_page_cost`/`seq_page_cost` = 4. Neste armazenamento (SSD virtualizado numa VM da Azure) não há penalidade de acesso aleatório; o valor 4 foi pensado para discos rotativos. Com razão próxima de 1, o planejador superestima o custo de acessos por índice e tende a preferir varreduras sequenciais mais cedo do que seria ideal; para SSD é comum usar valores perto de 1,1. A diferença de 15% a favor do aleatório está dentro da variação esperada de uma única medição. A medição é limitada: uma *thread*, `O_DIRECT` (sem *readahead* do sistema operacional) e uma passada de cada tipo.

Parte II

`relpages` e tuplas por página. Cada tupla ocupa 24 B de cabeçalho (23 B arredondados) mais 8 B de dados (dois inteiros), ou seja, 32 B, mais 4 B do ponteiro de linha na página: 36 B. Descontados os 24 B do cabeçalho da página, sobram 8.168 B: $\lfloor 8168/36 \rfloor = 226$ tuplas por página. Então $\lceil 100.000/226 \rceil = 443$ páginas, valor medido em `relpages` e em `pg_relation_size` (3.629.056 B / 8192). A média de 225,73 é 100.000/443: a última página fica incompleta.

Estatísticas e acesso à tabela. `k` é único (`n_distinct = -1`) e totalmente correlacionado com a ordem física (correlação 1,0). `v` tem 100 valores, cada um com frequência próxima de 1%, e correlação 0,006 com a ordem física, isto é, as linhas de um mesmo valor estão espalhadas por toda a tabela. Como a lista de valores comuns já cobre todos os 100 valores, não há histograma. Com 226 tuplas por página e 1% de linhas por valor, a fração de páginas com pelo menos uma linha com $v = 50$ é cerca de $1 - 0{,}99^{226} \approx 90\%$; por isso, sem índice, a consulta faz `Seq Scan` (443 blocos, todos já em `shared_buffers`, porque 3,5 MB cabem nos 128 MB), e com índice o plano é `Bitmap Heap Scan`: o índice não evita ler quase todas as páginas, e o *bitmap* apenas as lê em ordem física. Os dois contadores (`pg_statio_user_tables` e `EXPLAIN`) concordam: 443 blocos.

Altura do índice. Altura 1 significa que há um nível interno (a raiz) acima das folhas. Para 100.000 tuplas há 88 folhas e 90 páginas no total (metapágina + raiz + 88 folhas), então a raiz aponta diretamente para as folhas. Para 1.000.000 há 847 folhas e 854 páginas: descontadas a metapágina, sobram 6 páginas internas, a raiz e 5 intermediárias, cada uma com cerca de 170 ponteiros para folhas. As 847 folhas não cabem numa só página interna, então é preciso um nível a mais e a altura passa a 2. O número de folhas cresce quase linearmente (88 para 847, cerca de 9,6 vezes para 10 vezes mais tuplas), e a altura cresce com o logaritmo.

Fillfactor. Tamanho: só o 70 aumenta o índice (1084 páginas contra 854, ver D1). Altura: 2 em todos. Tempo: a criação ficou entre 883 e 1290 ms e a consulta entre 4,35 e 5,26 ms, sem diferença atribuível ao *fillfactor* numa execução única (ver D2). Um *fillfactor* menor deixa espaço livre nas folhas para inserções futuras sem divisão de páginas, ao custo de um índice maior.

Crescente e descendente. O índice descendente (`v DESC NULLS FIRST`) ficou idêntico em páginas, folhas, altura e densidade. Um índice B-tree pode ser percorrido nos dois sentidos, então para uma única coluna o sentido só importa em ordenações com colunas de direções mistas; como `v` não tem NULL, `NULLS FIRST` também não muda nada. O plano da igualdade foi o mesmo (`Bitmap Heap Scan`); as diferenças de tempo (T10: 4,82 a 7,13 ms; T9: 4,35 a 5,26 ms) são de uma única medição.

Parte III

Por que o índice de `votes` nem sempre é usado. O planejador escolhe o plano de menor custo estimado. Para `votes > 35000` (6 linhas) o índice custa 8,49 contra 38,05 do `Seq Scan`; para `votes > 1000` (82,2% da tabela) a leitura sequencial compensa. O limiar medido foi $X = 1550$ (36,7% das linhas), explicado em D3: numa tabela de 15 blocos, o índice só perde quando devolve uma fração grande das linhas.

T13. `MAX(votes)` é reescrito pelo PostgreSQL como "leia a última entrada do índice de `votes`" (`Index Only Scan` com `Limit 1`), executado uma vez; custo 35,37 e 0,11 ms. O `>= ALL` não tem essa reescrita: cada uma das 1.844 linhas é comparada com os valores da subconsulta materializada, cerca de 3,4 milhões de comparações no pior caso; custo 43.620,99 (cerca de 1.200 vezes o do `MAX`). O tempo real (1,24 ms, cerca de 11 vezes o do `MAX`) é bem menor que a razão de custos, pois o `>= ALL` para na primeira comparação que falha, o que o modelo de custo não prevê. Os dois devolvem o mesmo resultado.

T14. Subconsulta (custo 43,34) e auto-junção (49,49) devolvem as mesmas 9 linhas, mas os planos são diferentes: a subconsulta escalar é avaliada uma vez e vira constante para um `Index Scan`; a junção usa `Nested Loop` com dois `Index Scan`. O planejador não transforma uma na outra porque não pode provar que o título é único: `movie` tem 1.844 linhas e 1.833 títulos distintos (um título repetido faria a subconsulta falhar e a junção multiplicar linhas). Os tempos (0,096 ms e 0,025 ms) são pequenos demais para uma comparação confiável.

T15. Antes dos índices as quatro consultas faziam `Seq Scan`. Depois, `prefixo_like`, `prefixo_substr` e `sufixo_reverse` passaram a usar `Bitmap Index Scan` (custos 22,19; 22,23; 20,53, contra 38 a 43 do `Seq Scan`), e `sufixo_like` continuou em `Seq Scan` (38,05). Cada consulta só usa o índice cuja expressão coincide com a dela: `title LIKE 'C%'` usa o índice com `text_pattern_ops` (que compara byte a byte e permite transformar o prefixo numa faixa de busca; um índice comum não serve para `LIKE` em *collations* que não sejam "C"), `substr(title,1,1)` usa o índice sobre a expressão, e `reverse(title) LIKE 'I%'` usa o índice sobre `reverse(title)`. Um sufixo (`LIKE '%I'`) não corresponde a nenhuma faixa contígua do índice sobre `title`; o diagnóstico com `enable_seqscan = off` mostrou que ainda assim o plano é `Seq Scan` (custo $10^{10} + 38{,}05$, a penalidade do parâmetro), isto é, nenhum caminho por índice é possível. Reescrevê-lo como prefixo de `reverse(title)` resolve.

T16. O otimizador usa um histograma de 100 faixas com o mesmo número de linhas em cada faixa e supõe distribuição uniforme *dentro* de cada faixa. Em `votes < 1000` as faixas são estreitas (721, 771, 808, 844, ...), e a estimativa foi boa: 328 contra 326 reais. Em `votes > 35000` a última faixa vai de 28.641 a 53.567 votos e tem cerca de 18 linhas; a hipótese de uniformidade interpola linearmente e atribui cerca de 75% da faixa (aproximadamente 14 linhas) ao trecho acima de 35.000, quando as linhas reais se concentram perto do início dela (cauda longa): foram estimadas 12 e existem 6, erro de 2 vezes. Com `STATISTICS 1000` as faixas ficam muito mais estreitas, e as estimativas passam para 325 (contra 326) e 5 (contra 6).

Parte IV

Tempo total e tentativas. Com $k = 1$ todas as configurações levam cerca de 12,3 s, que é $120 \times 0{,}1$ s mais a sobrecarga. Em READ COMMITTED (a e b) o tempo cai para cerca de 1,7 s com $k = 10$ (speedup 7,4 e 7,3, contra 10 ideal) e a média de tentativas fica em 1,2: os poucos conflitos são agentes que sortearam o mesmo assento (26 e 21 casos de "assento tomado" com $k = 10$). Em a/SERIALIZABLE o tempo *não melhora*: fica em 12,3 a 12,6 s para qualquer k, e a média de tentativas é praticamente k (2,00; 3,98; 5,93; 7,87; 9,82), com um total de tentativas próximo de $120k$. Alguns clientes precisaram de 118 a 124 tentativas, quase uma por reserva concluída por outros: um agente pode ficar sem avançar enquanto os demais atendem a fila. Em b/SERIALIZABLE o tempo melhora (12,31 s para 3,42 s com $k = 10$, speedup 3,6), mas muito

menos que em READ COMMITTED, e as tentativas crescem (média 2,46 com $k = 10$).

Explicação pelo protocolo do PostgreSQL. Em READ COMMITTED cada comando enxerga um *snapshot* novo e as leituras não bloqueiam; o `UPDATE` bloqueia apenas a linha alterada, até o *commit*. Se outra transação já atualizou e confirmou o mesmo assento, o `UPDATE` reavalia a condição `disp = true` na versão nova da linha, não encontra nada e devolve zero linhas, que é o "assento tomado". Como nenhuma trava é mantida durante a espera de 0,1 s, a versão a e a versão b se comportam do mesmo modo. Em SERIALIZABLE o PostgreSQL usa SSI: registra com travas de predicado (SIREAD) o que cada transação leu e aborta com 40001 (*could not serialize access*) quando encontra um ciclo de dependências leitura-escrita entre transações concorrentes. Na versão a, cada transação lê a tabela inteira (`WHERE disp = true`, que implica varredura completa e uma trava no nível da relação), espera 0,1 s e então escreve em um assento; duas transações concorrentes se leem e se escrevem mutuamente, formando exatamente esse ciclo, e uma delas é abortada. Como os k agentes partem juntos e ficam concorrentes durante toda a espera, apenas uma transação por rodada de 0,1 s consegue confirmar, e as outras $k-1$ refazem. O trabalho fica serializado (cerca de $120 \times 0{,}1$ s $= 12$ s, qualquer que seja k) e as tentativas crescem com k. Na versão b as transações são curtas (milissegundos), e a espera ocorre fora de transação; a janela de concorrência é muito menor, o que reduz as falhas de 1.058 (a, $k = 10$) para 153 (b, $k = 10$), mas elas não desaparecem (ver D5).

Diferenças a × b e entre os níveis. Em READ COMMITTED não há diferença relevante entre a e b. Em SERIALIZABLE a versão b é muito melhor que a (3,42 s contra 12,62 s com $k = 10$), pois separa a espera das transações. Para qualquer versão, o READ COMMITTED é o mais rápido, e a garantia de não duplicar assentos é a mesma nos quatro casos, vinda do `UPDATE` condicional; o nível SERIALIZABLE, aqui, só acrescenta abortos.

9.6 Declaração de uso de ferramentas de IA

Ferramenta. Foi usado o Claude (Anthropic), em conversas na interface de chat, ao longo de todo o trabalho.

O que foi solicitado. (1) Análise do enunciado e do repositório-modelo e divisão do trabalho em etapas. (2) Escrita do código das tarefas: o assistente escreveu os programas `src/t01_sistema.py` a `src/t17_reservas.py`, com explicação do que cada um faz, incluindo as decisões descritas na Seção 9.1. (3) Escrita do texto de `docs/previsao.md`, elaborado pelo assistente a partir de raciocínio teórico (tamanho de tupla e de página, deduplicação em índices, custo de varredura e comportamento de transações) antes de qualquer medição; a estudante o colou no arquivo e o commitou. (4) Interpretação dos resultados e redação deste relatório (código em LaTeX), a partir dos arquivos de `resultados/` enviados pela estudante. As hipóteses não verificadas estão identificadas como tais no texto (D1 e D5).

O que a estudante fez. Criou o repositório, executou todos os comandos no Codespaces (`make up`, `params`, `load`, `parteI` a `parteIV` e `test`), colou o código nos arquivos, verificou a compilação, resolveu falhas de execução (uma falha inicial do Docker, que foi resolvida repetindo o comando), commitou e enviou as versões, e conferiu os resultados com o verificador. Além da remoção de linhas residuais de colagem (os comandos `cat` e `EOF`) em `docs/previsao.md` e nos arquivos de código, e de pequenos ajustes de formatação (um espaço em `t16_seletividade.py` e a quebra de linha no fim dos arquivos), o código fornecido pelo assistente não foi alterado.

Observação. O assistente acertou a maior parte das previsões, mas errou algumas (Seção 9.4: o *fillfactor*, o limiar de T12, a comparação entre as versões a e b em READ COMMITTED e o b/SERIALIZABLE); essas divergências foram mantidas e explicadas em vez de a previsão ser alterada depois.